Group 15 RBE 3001 Lab 5 Report

E. Scher
Robotics Engineering
Worcester Polytechnic Institute
Worcester, USA
ecscher@wpi.edu

S. Jeon
Robotics Engineering
Worcester Polytechnic Institute
Worcester, USA
sjeon@wpi.edu

A. Zilberleyb
Robotics Engineering
Worcester Polytechnic Institute
Worcester, USA
aszilberleyb@wpi.edu

***Abstract*—This report presents the final design and implementation of an autonomous robotic pick-and-place system for the WPI Robotics Engineering class RBE 3001: Unified Robotics III lab. Based on the OpenManipulator-X (OMX) serial robotic arm platform, this lab covers topics in forward, inverse, and differential kinematics, as well as computer vision and trajectory generation.**

I. Introduction

The objective of the final RBE 3001: Unified Robotics III lab was to compile everything from each previous lab into a single cohesive final project. The goal of this project was to take the OMX arm and combine it with a calibrated camera to pick up colored balls and put them in designated areas (Fig. 1).

Fig. 1. Testing communication between MATLAB and the OMX showing joint angle commands and responses.

II. Methodology

A. Establishing Serial Communication with OMX

To establish communication between MATLAB and the OMX, it was connected to a computer running Ubuntu 24.04 LTS via a USB-A to micro-USB cable. MATLAB R2025a was installed with the Dynamixel SDK properly configured in the system path to enable USB communication with the robot's servos.

To test communication, three methods were written to set the OMX's joint angles, interpolate to a new set of joint angles over time, and read the position and velocity of each joint.

The position-time graphs of the joint interpolation (Fig. 2) exhibit a Linear Segment with Parabolic Blends (LSPB, or Trapezoidal Velocity Profile) motion profile. The motion of the OMX in this test primarily served to verify that the Dynamixel servos were capable of the smooth motion required for the colored ball sorting application.

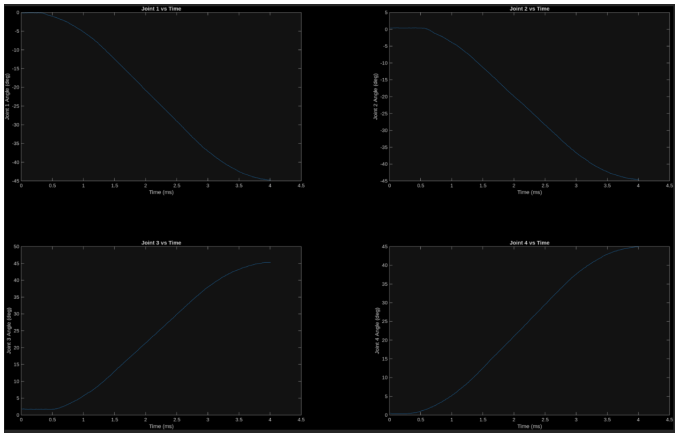

Fig. 2. OMX Arm Testing Environment.

B. Forward Kinematics Derivation

In order to move the arm to the balls, the forward kinematics had to be calculated for use in singularity detection and Jacobian calculation.

To solve the forward kinematics of the Open X robotic arm, the Denavit-Hartenberg (DH) convention was used to obtain a transformation matrix between each set of joints. Multiplying these matrices together yielded the final transformation from the base of the arm to the tip of the end effector.

The frames (Fig. 3) were chosen using the traditional DH assignment convention. The base frame (frame 0) was unconstrained, so the Z axis was assigned pointing vertically. For each subsequent joint, the Z axis was assigned pointing perpendicular to the axis of rotation, meaning that a counterclockwise rotation of the joint corresponded to a positive rotation about the Z axis. For the end effector frame, the decision for frame assignments is somewhat arbitrary, so frame 5 had its Z axis pointing in the same direction as frame 4 for simplicity. The X axis of each frame was assigned along the common normal from the previous frame to the

current frame. The link between joints 2 and 3 had a relative horizontal offset in addition to the vertical offset (Fig. 4). This required the calculation of the angle (θ) between the vertical portion of the link and the common normal between them, so a correct rotation about Z on frame 2 could be performed before translating to frame 3. The Y component simply completes the right-handed coordinate system for each frame.

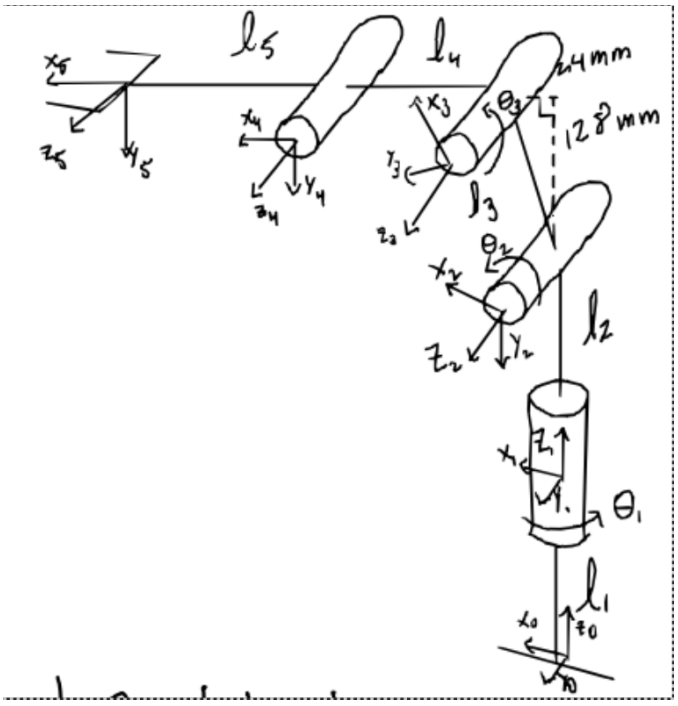

Fig. 3. OMX Arm Frame Assignments

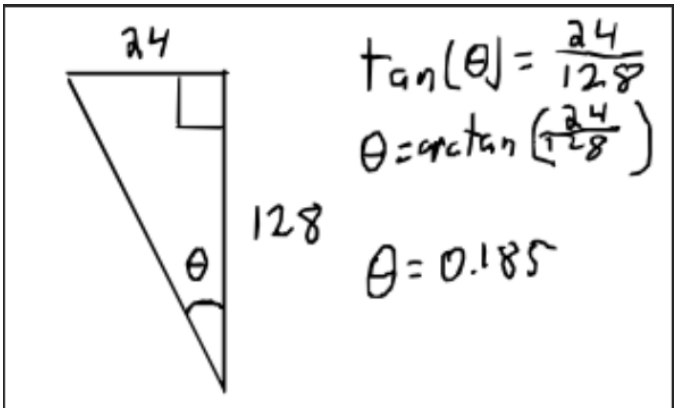

Fig. 4. Frame 2-3 Angle Offset Calculation

The DH parameters for each frame were found in the traditional order: rotation about Z, translation along Z, translation along X, and rotation about X.

After solving the Forward Kinematics of the arm, a MATLAB function called dh2mat was implemented to take a DH table and return a matrix representing the transformation between joints. Another function called dh2fk was implemented to represent the transformation from the first joint to the end effector.

TABLE I
DENAVIT–HARTENBERG PARAMETERS FOR THE OMX ARM

T_{i-1}^{i}	θ_z	d_z	a_x	α_x
1	0	l_1	0	0
2	θ_1^*	l_2	0	$-\frac{\pi}{2}$
3	$\theta_2^* - \frac{\pi}{2} + \theta_{offset}$	0	l_3	0
4	$\theta_3^* - \frac{\pi}{2} - \theta_{offset}$	0	l_4	0
5	θ_4^*	0	l_5	0

A symbolic expression was generated to represent a DH table with unknown variables. This table was then passed into the dh2fk function returning a symbolic forward kinematics matrix. To improve on this, the MATLAB Function was used to convert the symbolic matrix into a floating-point matrix, as using symbolic variables is considerably slower than floating point operations in MATLAB. Generally, symbolic variables are useful tools, but in the context of the DH tables, it could easily have been represented as a traditional floating-point function.

To test the DH function, a program defined 3 joint array angles which were sent to the robot. During these movements, the robot recorded the positions of each joint and ran them through the DH table to find the end effector position in operational space and graph them over time.

The robot was commanded to move in the X-Z plane to coordinates that roughly resembled a triangle. Fig. 5 shows the curved trajectory of the robot in operational space, while Fig. 6 shows the straight trajectory of each joint in configuration space.

The shape of the end effector trajectory should appear triangular, as the robot was only commanded to move to 3 points in the 2d XZ planar space. The lines in Fig. 6 appear triangular in nature, while the lines in Fig. 5 appear distorted and curved. This is due to the difference between plotting in configuration space and plotting in operational space. Configuration space, which is defined by the angles of the joints, shows an undistorted triangle. Each axis of Fig. 6 shows a different joint in configuration space, thus the position of those points in space are only controlled by one variable per axis. Conversely, in operation space, each point is defined by the forward kinematic calculation of the combination of all the joint angles. This is why the operational space graph appears distorted, as the compound position of all the joint angles over time do not necessarily trace a linear trajectory.

C. Inverse Kinematic Derivation

In order to convert the coordinates of the colored balls in 3d space to joint angles for the robot to run to, the inverse kinematics had to be solved.

To solve the inverse kinematics of the arm, the geometric approach was used with kinematic decoupling to simplify the solution process. The arm was simplified into a waist joint (θ_1), a two-link planar arm (shoulder joint θ_2 and elbow joint θ_3) and a wrist joint with the end effector attached (θ_4)

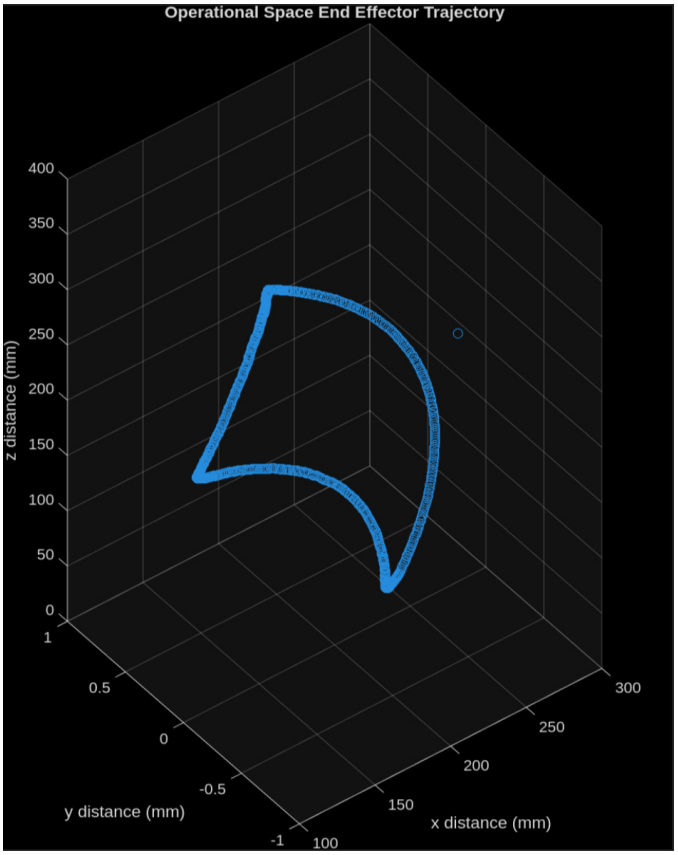

Fig. 5. Operational Space End Effector Trajectory

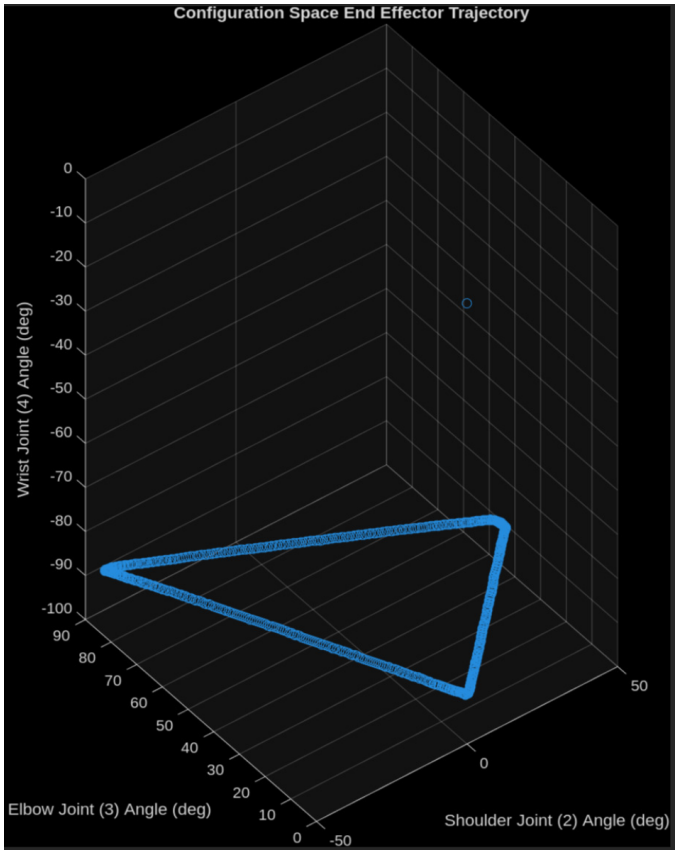

Fig. 6. Configuration Space End Effector Trajectory

As the OMX arm has 4 DOF, but is moving in 3d space, there are infinitely many solutions to the inverse kinematics of the arm at any point. To solve this, a fourth input DOF (γ) was added to constrain the wrist joint and collapse the infinite solutions of the IK arm to a finite solution set.

θ_1 was defined as the arc tangent of the end effector's y position over the end effector's x position:

$$\theta_1 = \arctan(py, px) \tag{1}$$

Solving the two-link planar arm component of the OMX derives the angles of the shoulder and elbow joints (θ_1 and θ_2) in terms of the location of the wrist joint in 3d space. This requires finding equations for the wrist in terms of the end effector positions.

$$p_{4_z} = p_z + (l_5 * \sin(\gamma)) \tag{2}$$

The z component of the wrist joint is influenced by the height component of the angle between the end effector and the ground ($l_5 * \sin(\gamma)$). As γ increases, more of the z component of the end effector link is incorporated into the wrist joint z coordinate.

$$p_{4_x} = p_x - (\cos(\theta_1) * l_5 * \cos(\gamma)) \tag{3}$$
$$p_{4_y} = p_y - (\sin(\theta_1) * l_5 * \cos(\gamma)) \tag{4}$$

The x and y component of the wrist joint are influenced by the length component of the angle between the end effector and the ground ($l_5 * \cos(\gamma)$). As γ increases, more of the x and y component of the end effector link is incorporated into the wrist joint's x and y coordinates respectively. Moreover, the waist joint influences how much of the length component of the end effector link is considered in the position of joint 4. This acts as a percentage of how much to incorporate γ.

To find the equations for the shoulder and elbow joints (θ_3 and θ_4) the OMX arm was simplified to use a two-link arm.

To derive θ_2 the alpha-beta-delta method was used to decompose the triangle formed by link 3, link 4, and the vector from the shoulder joint to the wrist joint ($\vec{p_4}$). θ_2 is composed of the sum of angles α and β. Angle α is the angle between $\vec{p_4}$ and the horizontal axis.

$$r = \sqrt{p_{4_x}^2 + p_{4_y}^2} \tag{5}$$
$$dz = p_{4_z} - (l_1 + l_2) \tag{6}$$
$$\alpha = atan2(dz, r) \tag{7}$$

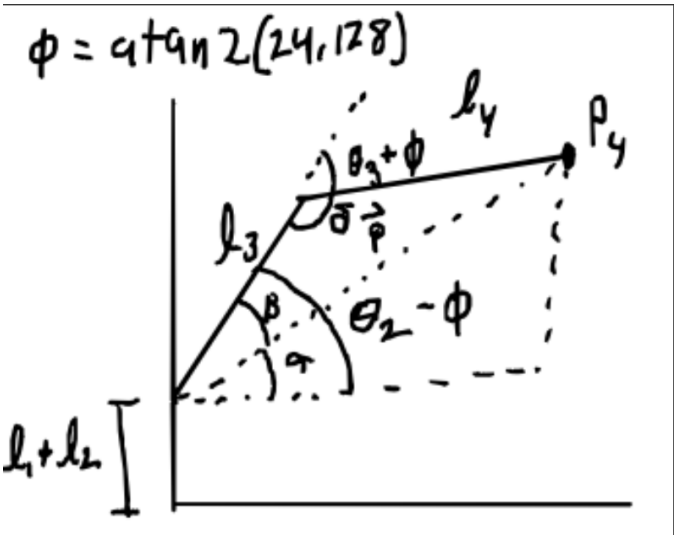

Fig. 7. OMX Two-Link Arm Component

r and dz together define α as the angle between $\vec{p_4}$ and the horizontal axis. β is defined using the law of cosines.

$$\beta = \arccos\left(\frac{\sqrt{r^2+dz^2}^2 + l_3^2 - l_4^2}{2 * l_3 * l_4}\right) \tag{8}$$

On the OMX, there exists an offset between the shoulder joint and elbow joint. This offset is defined as ϕ. θ_2 is the sum of α, β, and γ

$$\phi = atan2(24, 128) \tag{9}$$
$$\theta_2 = 90 - (\alpha + \beta + \phi) \tag{10}$$

θ_3 can be derived using the supplementary angle that $\theta_3 + \phi$ makes with the internal triangle angle defined as δ

$$\delta = arccos\left(\frac{l_3^2 + l_4^2 - \sqrt{r^2+dz^2}^2}{2 * l_3 * l_4}\right) \tag{11}$$
$$\theta_3 = 90 - (\delta - \phi) \tag{12}$$

θ_2 and θ_3 must be offset by 90 degrees to account for the rotational offset between the shoulder joint and the elbow and wrist joints. The wrist joint is simply the difference between γ and the sum of θ_2 and θ_3.

$$\theta_4 = \gamma - \theta_2 - \theta_3 \tag{13}$$

The final inverse kinematic equations for each joint are left in terms of the input variables x, y, z, and γ

$$\theta_1 = \arctan(py, px) \tag{14}$$
$$\theta_2 = 90 - (\alpha + \beta + \phi) \tag{15}$$
$$\theta_3 = 90 - (\delta - \phi) \tag{16}$$
$$\theta_4 = \gamma - \theta_2 - \theta_3 \tag{17}$$

As there is a two-link arm component in the OMX, naturally there are also multiple solutions to each inverse kinematic solve. As the OMX has very little range of motion in the elbow down direction, the heuristic of always using the elbow up solution was chosen as a "tiebreaker".

Testing the inverse kinematic model was verified through running a set of arbitrary joint positions through the forward kinematic model to get the operational space coordinates, which were then run back through the inverse kinematics to get the configuration space joint angles. The initial joint angles matched the joint angles generated by the inverse kinematics.

D. Trajectory Generation

To follow operational space trajectories, cubic and quintic splines were generated using MATLAB.

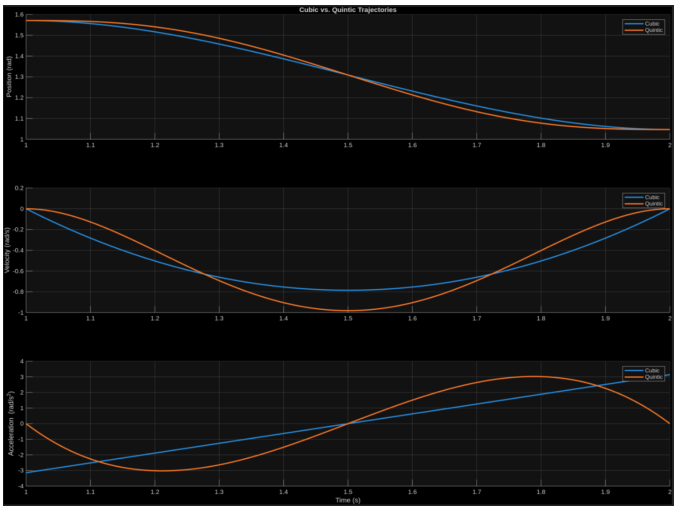

Fig. 8. Simplified 1 Dimensional Cubic and Quintic Trajectory Examples

Cubic and quintic splines are similar in function but differ slightly in shape and ability. Quintic splines offer smoother transitions by explicitly constraining both velocity and acceleration to zero at the endpoints, whereas cubic splines only control position and velocity. This difference is most visible in the acceleration plot, where the cubic trajectory exhibits a more gradual change, while the quintic trajectory curves sharply to enforce the acceleration conditions.

Cubic and Quintic splines were generated between three points in operational space, and during movement, the OMX followed the trajectories while collecting position data. The results showed that the operational space trajectories were the exact opposite of the configuration space trajectories followed using forward kinematic joint manipulation (Fig. 9).

E. Differential Kinematics Derivation and Singularity Detection

The differential kinematics of the OMX allowed for easy detection of singular points where the arm would fall behind its setpoint, causing undefined behavior.

The derivation of the Manipulator Jacobian was split into two parts. The translational component of the Jacobian was

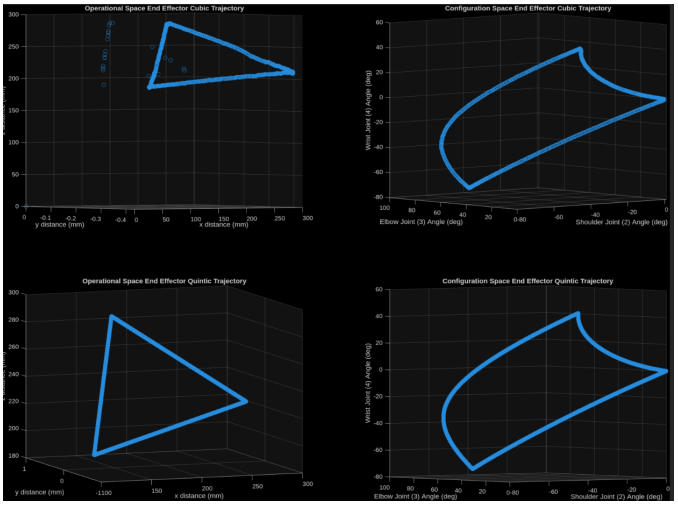

Fig. 9. Operational Space vs Configuration Space of Operational Space Cubic and Quintic Trajectories

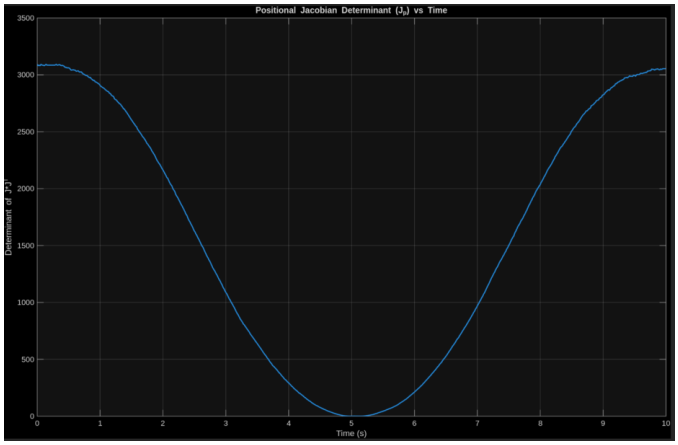

Fig. 10. Manipulator Jacobian Determinants vs Time

computed by taking the partial derivatives, with respect to each joint angle, of the translational component of the generated forward kinematics homogeneous transformation matrix generated by the DH table. The rotational component of the Jacobian was derived from the Z axis component of each successive homogeneous transformation matrix generated through successive multiplication of each row of the DH table. Combining the translational and rotational components together yields the full Manipulator Jacobian.

The matrix was run through the MATLAB Function method, which converted it to a floating-point function which runs much faster and is far less computationally expensive. This method saves the new floating-point function to a separate file which is called by the jacob3001 method in Robot.m. This function is then called by the dk3001 method which in turn, calculates the 6x1 matrix representing the end effector translational and rotational velocities in operational space.

To test the accuracy of the Manipulator Jacobian, a method to calculate the determinant of the Jacobian was written. This method returned the determinant of the current OMX position's Jacobian, as well as whether the Jacobian's determinant was less than a supplied threshold. This determined whether the position was a singularity (Fig. 10).

Singularity detection was tested by generating a cubic trajectory to run through the singular point above the base. At the singular point, the robot position's determinant would cross a threshold and the singularity was detected. To test the differential kinematics, the robot was told to move in a series of trajectories to form a triangular shape in operational space.

During the singularity test, the determinant of the Manipulator Jacobian falls to 0 at any singular points. This occurs because singular points are the result of a loss of an input or output DOF, which is represented by a full row or column of the Manipulator Jacobian, evaluated at that position, being 0. Mathematically, this results in a zero determinant because it makes the rows/columns linearly dependent, which drives the determinant to 0. Compared with configuration space control, singularities are far more hazardous to operational space control. With configuration space control, the joint angles are inputted directly to the robot, rather than running an operational space point through a solver.

The velocity vs time graphs of the end effector velocity over time (Fig. 11) match well with their respective trajectories. The two biggest features of note on the graphs are the noise of the graphs at high movement times, and the phase lag of the robot at the lower movement times. For t = 7 and t = 5, even though the measured velocities track the setpoints well, there is a significant amount of noise in the readings. This could be due to multiple factors, but the most likely is oscillation of the motors while following a trajectory at a slow enough speed where the oscillations and the actual movement are on roughly the same order of magnitude. This contrasts the graphs of velocity where t = 3 and t = 1, where the noise is virtually gone. Likely due to the high speed of travel of the end effector, the noise has an inconsequential effect on the overall graph. With the faster trajectories, there is a significant phase lag between the setpoint and the measured joint velocity. This is likely due to the trajectory commanding speeds that are physically faster than the robot can realize, resulting in the robot always playing "catch up" with the setpoint. Overall, t = 3 seems to be a good balance between noisy velocity data and phase lag.

F. Camera Intrinsic Calibration

To detect the balls on the board, a USB webcam was mounted to a post adjacent to the board. This camera had a fisheye lens, which significantly distorted the images and made the checkerboard appear curved. To combat this, MATLAB's Image Processing Toolbox was used to calibrate the camera and undistort the images it took. To undistort images, the camera's intrinsic matrix was found using MATLAB's camera calibrator. 80 images were captured using the webcam from multiple different positions and orientations around the checkerboard. The calibrator then used a corner detection

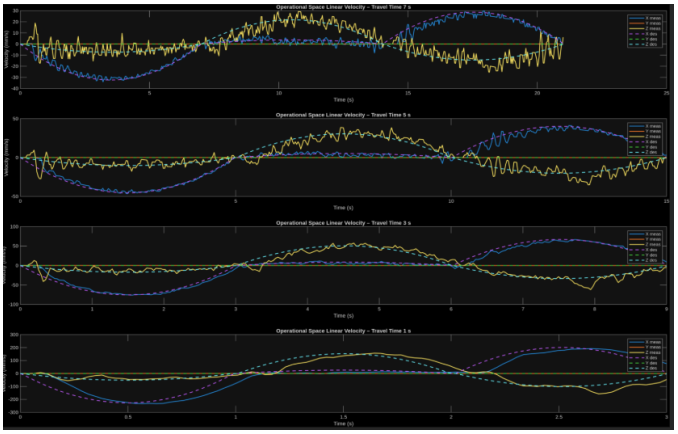

Fig. 11. Operational Space Velocity vs Time

algorithm to find the corners of each checker square and the X and Y axis of the board (Fig. 12).

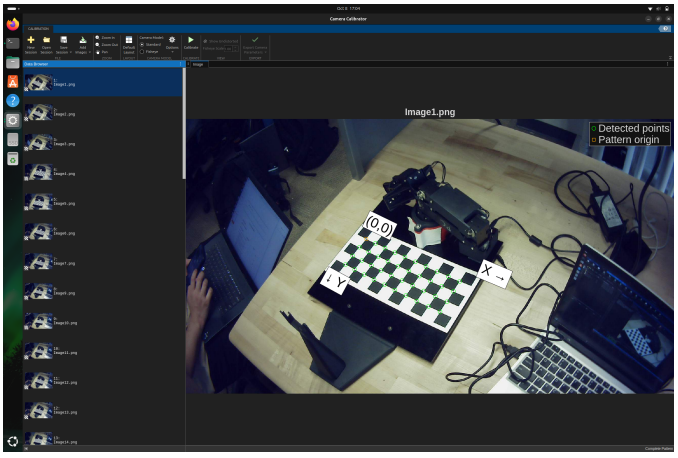

Fig. 12. Checkerboard Images and Corner Detection

MATLAB then ran a numerical optimization on the images to minimize the reprojection error from the image plane to world coordinates (Fig. 13). The results of the calibration were the camera matrix and distortion coefficients, which were used to undistort the images from the camera.

G. Camera Extrinsic Calibration

In order for the balls image plane coordinates to be converted to coordinates of the checkerboard, the camera's extrinsic matrix was solved for (Fig. 14).

H. Ball Color Masks

To determine the color of each ball, a color masks were generated using the HSV configurator in MATLAB's Color Thresholder tool. This resulted in being able to apply certain colors to the camera images to find balls of just certain colors (Fig. 15).

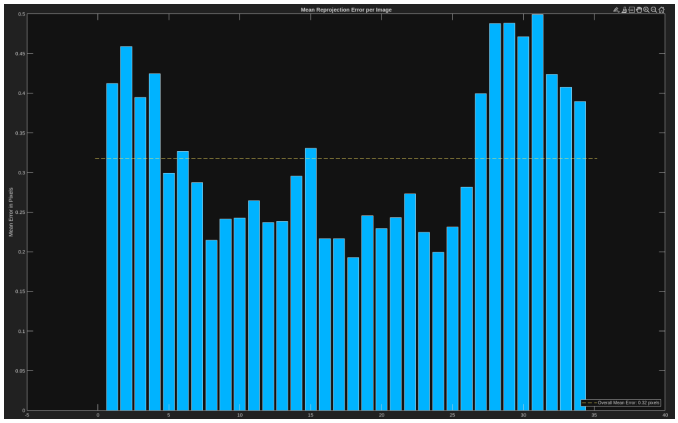

Fig. 13. Reprojection Error

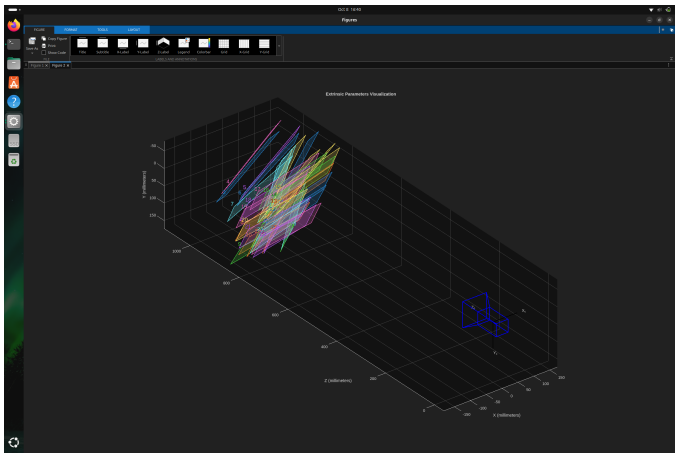

Fig. 14. Extrinsic Camera Calibration

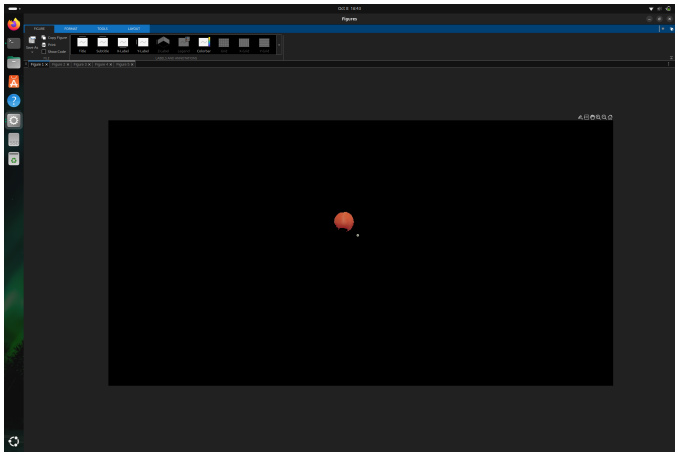

Fig. 15. Color Mask

I. Centroid Detector

To pick out where the balls were in the image, clusters of pixels were identified through their black/white color masked images, and their centroids and areas were found (Fig. 16).

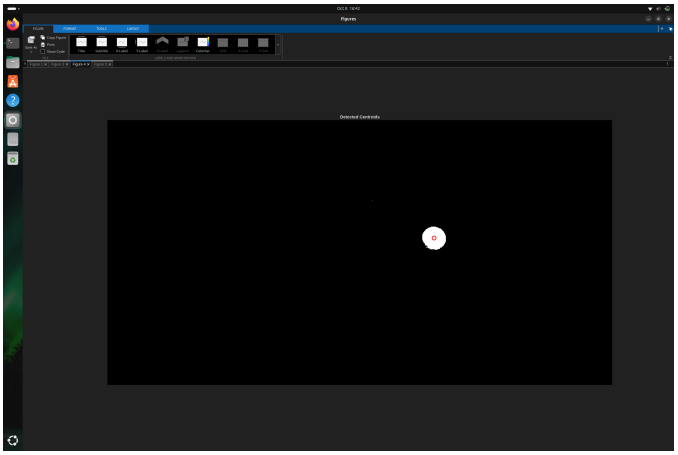

Fig. 16. Centroid Detection

J. Centroid Coordinate Corrector

As the balls are not flat on the checkerboard, the camera inherently sees a projection of the ball on the checkerboard. This is problematic due to the fact that the balls have a height, which means that if the arm attempted to pick up a ball, it would aim for a point behind the ball. To solve this, a centroid corrector was implemented to make the arm aim at the actual center of the ball on the board instead of the projection of the ball on the board Fig. 17.

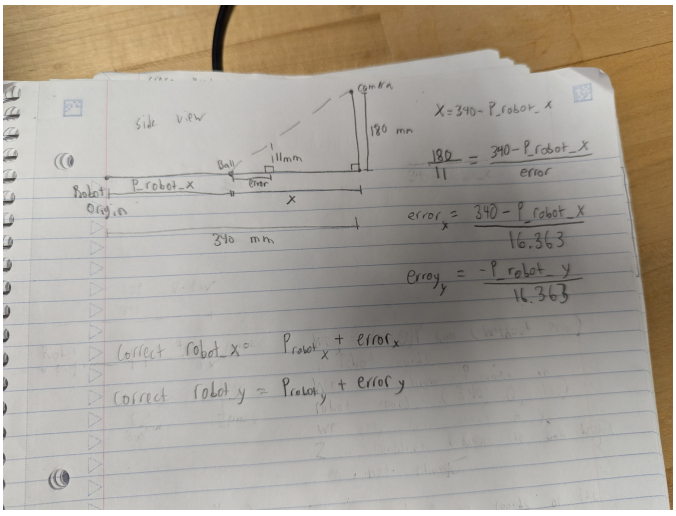

Fig. 17. Centroid Correction

K. Full Pipeline

The full program combined all of the image processing techniques into one cohesive pipeline. The images captured by the camera were run through the intrinsic camera matrix to account for distortion. After which, each color ball's color mask was applied to the image to isolate each ball individually. The centroids of the balls were found and the coordinates of the centroid projections were corrected to account for the height of the ball in the image. Finally, the corrected centroid coordinates were transformed into the reference frame of the base frame of the OMX arm. The arm was then moved to pick up the ball using the following function:

- Take image and run through full pipeline
- Move over the target ball
- Open gripper
- Move straight down over the ball
- Close gripper
- Move to ball color's dedicated receptacle
- Open gripper
- Repeat until no balls in image

III. Results

The complete system was tested using four colored balls placed randomly within the camera's field of view. The arm successfully identified, picked, and sorted all balls into their designated bins without human intervention. The overall pick-and-place success rate was 97% across 35 full ball cycles. The cycle time for each arm movement was set to 1 second, meaning each ball cycle took 3.5 seconds to complete. This speed was primarily limited by joint velocity constraints and gripper delay. The final integrated system demonstrated robust performance and accurate coordination between vision and kinematic control.

IV. Discussion and Conclusion

The integration of forward and inverse kinematics, trajectory generation, differential kinematics, and computer vision resulted in a fully autonomous pick-and-place robotic system using the OMX. The successful implementation of this pipeline demonstrates a comprehensive understanding of robotic motion planning, control, and computer vision.

The results confirmed that both the kinematic and vision subsystems performed as intended. The forward and inverse kinematic derivations accurately translated between configuration and operational space, with only minor deviations near singularities or at the edges of the workspace. The manipulator Jacobian provided a reliable means for singularity detection, allowing the robot to avoid configurations where control would otherwise become unstable. This was particularly important in operational space control, where small deviations in joint space can produce large error in end effector positioning.

The trajectory generation experiments highlighted key trade-offs between smoothness and speed. Quintic splines produced smoother motion profiles with reduced velocity discontinuities, while cubic splines allowed for faster but slightly jerkier motion. In practice, the robot's physical constraints made the quintic trajectories more suitable for precise movements such as grasping, while cubic trajectories were better suited for transit motions between tasks. The velocity-phase lag observed at high speeds showed the importance of matching

commanded trajectory speeds with the physical velocity limits of the Dynamixel servo motors.

The vision system, consisting of intrinsic and extrinsic camera calibration, color masking, and centroid correction, effectively localized the colored balls within the robot's frame of reference. The centroid correction was critical for accurate grasping, compensating for the height-induced projection error inherent in camera imaging. The final perception-to-action pipeline demonstrated strong reliability, with an overall pick accuracy of 97%. The small number of missed detections primarily resulted from small deviation in end effector positioning in operational space, suggesting that further tuning of trajectory following could be beneficial to reliability.

Overall, this project successfully demonstrated the integration of key robotic principles into a single cohesive system, and represented the culmination of all RBE 3001 concepts. The project not only validated the derived kinematic and vision models but also demonstrated their real-world applicability in autonomous manipulation tasks.

APPENDIX A
SUPPLEMENTARY MATERIALS

The source code and demonstration video for this project are available online:

- **Code Repository:** https://github.com/WPI-RBE-300n/RBE3001_A25_15/releases/tag/Lab_5
- **Demonstration Video:** https://drive.google.com/file/d/1pTpZVJRVZh20B1nUxeszxlBb7t4i6d_N/view